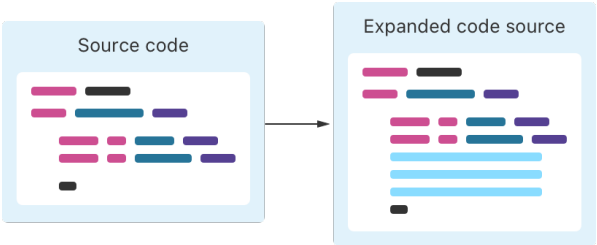
Macros

Use macros to generate code at compile time.

Macros transform your source code when you compile it, letting you avoid writing repetitive code by hand. During compilation, Swift expands any macros in your code before building your code as usual.



Expanding a macro is always an additive operation: Macros add new code, but they never delete or modify existing code.

Both the input to a macro and the output of macro expansion are checked to ensure they're syntactically valid Swift code. Likewise, the values you pass to a macro and the values in code generated by a macro are checked to ensure they have the correct types. In addition, if the macro's implementation encounters an error when expanding that macro, the compiler treats this as a compilation error. These guarantees make it easier to reason about code that uses macros, and they make it easier to identify issues like using a macro incorrectly or a macro implementation that has a bug.

Swift has two kinds of macros:

- *Freestanding macros* appear on their own, without being attached to a declaration.
- *Attached macros* modify the declaration that they're attached to.

You call attached and freestanding macros slightly differently, but they both follow the same model for macro expansion, and you implement them both using the same approach. The following sections describe both kinds of macros in more detail.

Freestanding Macros

To call a freestanding macro, you write a number sign (#) before its name, and you write any arguments to the macro in parentheses after its name. For example:

```
func myFunction() {
    print("Currently running \(#function)")
    #warning("Something's wrong")
}
```

In the first line, `#function` calls the `function()` macro from the Swift standard library. When you compile this code, Swift calls that macro's implementation, which replaces `#function` with the name

of the current function. When you run this code and call `myFunction()`, it prints “Currently running myFunction()”. In the second line, `#warning` calls the `warning(_:)` macro from the Swift standard library to produce a custom compile-time warning.

Freestanding macros can produce a value, like `#function` does, or they can perform an action at compile time, like `#warning` does.

Attached Macros

To call an attached macro, you write an at sign (@) before its name, and you write any arguments to the macro in parentheses after its name.

Attached macros modify the declaration that they’re attached to. They add code to that declaration, like defining a new method or adding conformance to a protocol.

For example, consider the following code that doesn’t use macros:

```
struct SundaeToppings: OptionSet {
    let rawValue: Int
    static let nuts = SundaeToppings(rawValue: 1 << 0)
    static let cherry = SundaeToppings(rawValue: 1 << 1)
    static let fudge = SundaeToppings(rawValue: 1 << 2)
}
```

In this code, each of the options in the `SundaeToppings` option set includes a call to the initializer, which is repetitive and manual. It would be easy to make a mistake when adding a new option, like typing the wrong number at the end of the line.

Here’s a version of this code that uses a macro instead:

```
@OptionSet<Int>
struct SundaeToppings {
    private enum Options: Int {
        case nuts
        case cherry
        case fudge
    }
}
```

This version of `SundaeToppings` calls an `@OptionSet` macro. The macro reads the list of cases in the private enumeration, generates the list of constants for each option, and adds a conformance to the `OptionSet` protocol.

For comparison, here’s what the expanded version of the `@OptionSet` macro looks like. You don’t write this code, and you would see it only if you specifically asked Swift to show the macro’s expansion.

```
struct SundaeToppings {
    private enum Options: Int {
        case nuts
        case cherry
        case fudge
    }

    typealias RawValue = Int
    var rawValue: RawValue
    init() { self.rawValue = 0 }
    init(rawValue: RawValue) { self.rawValue = rawValue }
    static let nuts: Self = Self(rawValue: 1 << Options.nuts.rawValue)
    static let cherry: Self = Self(rawValue: 1 << Options.cherry.rawValue)
    static let fudge: Self = Self(rawValue: 1 << Options.fudge.rawValue)
}
extension SundaeToppings: OptionSet { }
```

All of the code after the private enumeration comes from the `@OptionSet` macro. The version of `SundaeToppings` that uses a macro to generate all of the static variables is easier to read and easier to maintain than the manually coded version, earlier.

Macro Declarations

In most Swift code, when you implement a symbol, like a function or type, there's no separate declaration. However, for macros, the declaration and implementation are separate. A macro's declaration contains its name, the parameters it takes, where it can be used, and what kind of code it generates. A macro's implementation contains the code that expands the macro by generating Swift code.

You introduce a macro declaration with the `macro` keyword. For example, here's part of the declaration for the `@OptionSet` macro used in the previous example:

```
public macro OptionSet<RawType>() =
        #externalMacro(module: "SwiftMacros", type: "OptionSetMacro")
```

The first line specifies the macro's name and its arguments — the name is `OptionSet`, and it doesn't take any arguments. The second line uses the `externalMacro(module:type:)` macro from the Swift standard library to tell Swift where the macro's implementation is located. In this case, the `SwiftMacros` module contains a type named `OptionSetMacro`, which implements the `@OptionSet` macro.

Because `OptionSet` is an attached macro, its name uses upper camel case, like the names for structures and classes. Freestanding macros have lower camel case names, like the names for variables and functions.

> **Note**
>
> Macros are always declared as `public`. Because the code that declares a macro is in a different module from code that uses that macro, there isn't anywhere you could apply a nonpublic macro.

A macro declaration defines the macro's *roles* — the places in source code where that macro can be called, and the kinds of code the macro can generate. Every macro has one or more roles, which you write as part of the attributes at the beginning of the macro declaration. Here's a bit more of the declaration for `@OptionSet`, including the attributes for its roles:

```
@attached(member)
@attached(extension, conformances: OptionSet)
public macro OptionSet<RawType>() =
        #externalMacro(module: "SwiftMacros", type: "OptionSetMacro")
```

The `@attached` attribute appears twice in this declaration, once for each macro role. The first use, `@attached(member)`, indicates that the macro adds new members to the type you apply it to. The `@OptionSet` macro adds an `init(rawValue:)` initializer that's required by the `OptionSet` protocol, as well as some additional members. The second use, `@attached(extension, conformances: OptionSet)`, tells you that `@OptionSet` adds conformance to the `OptionSet` protocol. The `@OptionSet` macro extends the type that you apply the macro to, to add conformance to the `OptionSet` protocol.

For a freestanding macro, you write the `@freestanding` attribute to specify its role:

```
@freestanding(expression)
public macro line<T: ExpressibleByIntegerLiteral>() -> T =
        /* ... location of the macro implementation... */
```

The `#line` macro above has the `expression` role. An expression macro produces a value, or performs a compile-time action like generating a warning.

In addition to the macro's role, a macro's declaration provides information about the names of the symbols that the macro generates. When a macro declaration provides a list of names, it's guaranteed to produce only declarations that use those names, which helps you understand and debug the generated code. Here's the full declaration of `@OptionSet`:

```
@attached(member, names: named(RawValue), named(rawValue),
        named(`init`), arbitrary)
@attached(extension, conformances: OptionSet)
public macro OptionSet<RawType>() =
        #externalMacro(module: "SwiftMacros", type: "OptionSetMacro")
```

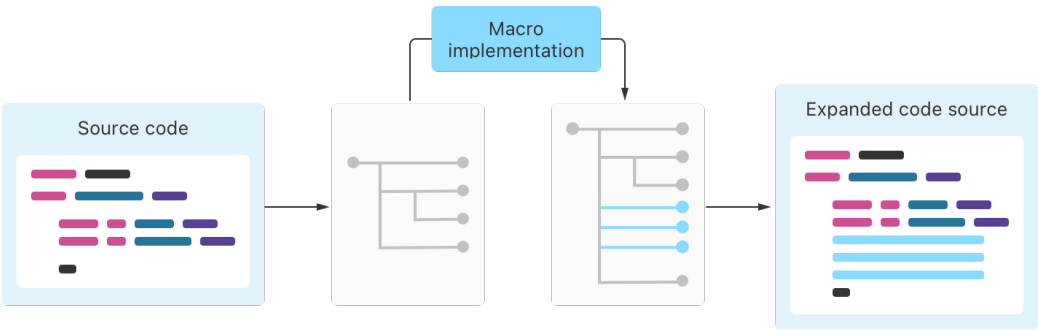
In the declaration above, the `@attached(member)` macro includes arguments after the `names:` label for each of the symbols that the `@OptionSet` macro generates. The macro adds declarations for symbols named `RawValue`, `rawValue`, and `init` — because those names are known ahead of time, the macro declaration lists them explicitly.

The macro declaration also includes `arbitrary` after the list of names, allowing the macro to generate declarations whose names aren't known until you use the macro. For example, when the `@OptionSet` macro is applied to the `SundaeToppings` above, it generates type properties that correspond to the enumeration cases, `nuts`, `cherry`, and `fudge`.

For more information, including a full list of macro roles, see attached and freestanding in Attributes.

Macro Expansion

When building Swift code that uses macros, the compiler calls the macros' implementation to expand them.



Specifically, Swift expands macros in the following way:

1. The compiler reads the code, creating an in-memory representation of the syntax.
2. The compiler sends part of the in-memory representation to the macro implementation, which expands the macro.
3. The compiler replaces the macro call with its expanded form.
4. The compiler continues with compilation, using the expanded source code.

To go through the specific steps, consider the following:

```
let magicNumber = #fourCharacterCode("ABCD")
```

The `#fourCharacterCode` macro takes a string that's four characters long and returns an unsigned 32-bit integer that corresponds to the ASCII values in the string joined together. Some file formats use integers like this to identify data because they're compact but still readable in a debugger. The Implementing a Macro section below shows how to implement this macro.

To expand the macros in the code above, the compiler reads the Swift file and creates an in-memory representation of that code known as an *abstract syntax tree*, or AST. The AST makes the code's structure explicit, which makes it easier to write code that interacts with that structure — like a compiler or a macro implementation. Here's a representation of the AST for the code above, slightly simplified by omitting some extra detail:

- Constant declaration
  - Constant name: `magicNumber`
  - Constant value
    - Macro call
      - Macro name: `fourCharacterCode`
      - Macro arguments:
        - String literal: `"ABCD"`

The diagram above shows how the structure of this code is represented in memory. Each element in the AST corresponds to a part of the source code. The "Constant declaration" AST element has two child elements under it, which represent the two parts of a constant declaration: its name and its value. The "Macro call" element has child elements that represent the macro's name and the list of arguments being passed to the macro.

As part of constructing this AST, the compiler checks that the source code is valid Swift. For example, `#fourCharacterCode` takes a single argument, which must be a string. If you tried to pass an integer argument, or forgot the quotation mark (") at the end of the string literal, you'd get an error at this point in the process.

The compiler finds the places in the code where you call a macro, and loads the external binary that implements those macros. For each macro call, the compiler passes part of the AST to that macro's implementation. Here's a representation of that partial AST:

- Macro call
  - Macro name: `fourCharacterCode`
  - Macro arguments
    - String literal: `"ABCD"`

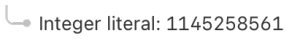
The implementation of the `#fourCharacterCode` macro reads this partial AST as its input when expanding the macro. A macro's implementation operates only on the partial AST that it receives as its input, meaning a macro always expands the same way regardless of what code comes before and after it. This limitation helps make macro expansion easier to understand, and helps your code build faster because Swift can avoid expanding macros that haven't changed.

Swift helps macro authors avoid accidentally reading other input by restricting the code that implements macros:

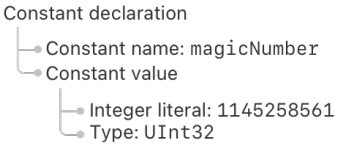
- The AST passed to a macro implementation contains only the AST elements that represent the macro, not any of the code that comes before or after it.
- The macro implementation runs in a sandboxed environment that prevents it from accessing the file system or the network.

In addition to these safeguards, the macro's author is responsible for not reading or modifying anything outside of the macro's inputs. For example, a macro's expansion must not depend on the current time of day.

The implementation of `#fourCharacterCode` generates a new AST containing the expanded code. Here's what that code returns to the compiler:



When the compiler receives this expansion, it replaces the AST element that contains the macro call with the element that contains the macro's expansion. After macro expansion, the compiler checks again to ensure the program is still syntactically valid Swift and all the types are correct. That produces a final AST that can be compiled as usual:



This AST corresponds to Swift code like this:

```
let magicNumber = 1145258561 as UInt32
```

In this example, the input source code has only one macro, but a real program could have several instances of the same macro and several calls to different macros. The compiler expands macros one at a time.

If one macro appears inside another, the outer macro is expanded first — this lets the outer macro modify the inner macro before it's expanded.

Implementing a Macro

To implement a macro, you make two components: A type that performs the macro expansion, and a library that declares the macro to expose it as API. These parts are built separately from code that uses the macro, even if you're developing the macro and its clients together, because the macro implementation runs as part of building the macro's clients.

To create a new macro using Swift Package Manager, run `swift package init --type macro` — this creates several files, including a template for a macro implementation and declaration.

To add macros to an existing project, edit the beginning of your `Package.swift` file as follows:

- Set a Swift tools version of 5.9 or later in the `swift-tools-version` comment.
- Import the `CompilerPluginSupport` module.

- Include macOS 10.15 as a minimum deployment target in the `platforms` list.

The code below shows the beginning of an example `Package.swift` file.

```
// swift-tools-version: 5.9

import PackageDescription
import CompilerPluginSupport

let package = Package(
    name: "MyPackage",
    platforms: [ .iOS(.v17), .macOS(.v13)],
    // ...
)
```

Next, add a target for the macro implementation and a target for the macro library to your existing `Package.swift` file. For example, you can add something like the following, changing the names to match your project:

```
targets: [
    // Macro implementation that performs the source transformations.
    .macro(
        name: "MyProjectMacros",
        dependencies: [
            .product(name: "SwiftSyntaxMacros", package: "swift-syntax"),
            .product(name: "SwiftCompilerPlugin", package: "swift-syntax")
        ]
    ),

    // Library that exposes a macro as part of its API.
    .target(name: "MyProject", dependencies: ["MyProjectMacros"]),
]
```

The code above defines two targets: `MyProjectMacros` contains the implementation of the macros, and `MyProject` makes those macros available.

The implementation of a macro uses the SwiftSyntax module to interact with Swift code in a structured way, using an AST. If you created a new macro package with Swift Package Manager, the generated `Package.swift` file automatically includes a dependency on SwiftSyntax. If you're adding macros to an existing project, add a dependency on SwiftSyntax in your `Package.swift` file:

```
dependencies: [
    .package(url: "https://github.com/swiftlang/swift-syntax", from: "509.0.0")
],
```

Depending on your macro's role, there's a corresponding protocol from SwiftSyntax that the macro implementation conforms to. For example, consider `#fourCharacterCode` from the previous section. Here's a structure that implements that macro:

```swift
import SwiftSyntax
import SwiftSyntaxMacros

public struct FourCharacterCode: ExpressionMacro {
    public static func expansion(
        of node: some FreestandingMacroExpansionSyntax,
        in context: some MacroExpansionContext
    ) throws -> ExprSyntax {
        guard let argument = node.argumentList.first?.expression,
              let segments = argument.as(StringLiteralExprSyntax.self)?.segments,
              segments.count == 1,
              case .stringSegment(let literalSegment)? = segments.first
        else {
            throw CustomError.message("Need a static string")
        }

        let string = literalSegment.content.text
        guard let result = fourCharacterCode(for: string) else {
            throw CustomError.message("Invalid four-character code")
        }

        return "\(raw: result) as UInt32"
    }
}

private func fourCharacterCode(for characters: String) -> UInt32? {
    guard characters.count == 4 else { return nil }

    var result: UInt32 = 0
    for character in characters {
        result = result << 8
        guard let asciiValue = character.asciiValue else { return nil }
        result += UInt32(asciiValue)
    }
    return result
}
enum CustomError: Error { case message(String) }
```

If you're adding this macro to an existing Swift Package Manager project, add a type that acts as the entry point for the macro target and lists the macros that the target defines:

```swift
import SwiftCompilerPlugin

@main
struct MyProjectMacros: CompilerPlugin {
    var providingMacros: [Macro.Type] = [FourCharacterCode.self]
}
```

The `#fourCharacterCode` macro is a freestanding macro that produces an expression, so the `FourCharacterCode` type that implements it conforms to the `ExpressionMacro` protocol. The `ExpressionMacro` protocol has one requirement, an `expansion(of:in:)` method that expands the AST. For the list of macro roles and their corresponding SwiftSyntax protocols, see attached and freestanding in Attributes.

To expand the `#fourCharacterCode` macro, Swift sends the AST for the code that uses this macro to the library that contains the macro implementation. Inside the library, Swift calls `FourCharacter`

`Code.expansion(of:in:)`, passing in the AST and the context as arguments to the method. The implementation of `expansion(of:in:)` finds the string that was passed as an argument to `#fourCharacterCode` and calculates the corresponding 32-bit unsigned integer literal value.

In the example above, the first `guard` block extracts the string literal from the AST, assigning that AST element to `literalSegment`. The second `guard` block calls the private `fourCharacterCode(for:)` function. Both of these blocks throw an error if the macro is used incorrectly — the error message becomes a compiler error at the malformed call site. For example, if you try to call the macro as `#fourCharacterCode("AB" + "CD")` the compiler shows the error “Need a static string”.

The `expansion(of:in:)` method returns an instance of `ExprSyntax`, a type from SwiftSyntax that represents an expression in an AST. Because this type conforms to the `StringLiteralConvertible` protocol, the macro implementation uses a string literal as a lightweight syntax to create its result. All of the SwiftSyntax types that you return from a macro implementation conform to `StringLiteralConvertible`, so you can use this approach when implementing any kind of macro.

Developing and Debugging Macros

Macros are well suited to development using tests: They transform one AST into another AST without depending on any external state, and without making changes to any external state. In addition, you can create syntax nodes from a string literal, which simplifies setting up the input for a test. You can also read the `description` property of an AST to get a string to compare against an expected value. For example, here’s a test of the `#fourCharacterCode` macro from previous sections:

```
let source: SourceFileSyntax =
    """
    let abcd = #fourCharacterCode("ABCD")
    """

let file = BasicMacroExpansionContext.KnownSourceFile(
    moduleName: "MyModule",
    fullFilePath: "test.swift"
)

let context = BasicMacroExpansionContext(sourceFiles: [source: file])

let transformedSF = source.expand(
    macros:["fourCharacterCode": FourCharacterCode.self],
    in: context
)

let expectedDescription =
    """
    let abcd = 1145258561 as UInt32
    """

precondition(transformedSF.description == expectedDescription)
```

The example above tests the macro using a precondition, but you could use a testing framework instead.

Copyright © 2014–2025 Apple Inc. and the Swift project authors. All rights reserved.
This document is made available under a Creative Commons Attribution 4.0 International (CC BY 4.0) License.
Swift and the Swift logo are trademarks of Apple Inc.
Privacy Policy | Cookies